

alexandre.plana.pro@gmail.com
+33 6 71 03 97 84

SKILLS

Web ecosystem
Vue 3 · Pinia · single-spa · React · Vite · Dockview · shadcn · MapLibre · deck.gl · Socket.IO · Tailwind · HTML / CSS · WebGL

Languages
TypeScript · Go · C# · Python

Services & interoperability
API REST · MQTT · AIS · NMEA · CoT · MAVLink

Quality & tools
Jest · Vitest · SonarQube · GitLab CI · Docker · Git · Postman · Jira · Figma

Real-time 3D & XR
Unity 3D · C# · VR / AR · WPF · SolidWorks · 3ds Max

EDUCATION

React · TypeScript
Datacorp · 2026

TypeScript · Docker · Vue.js
Datacorp · 2023

Polymorph
Final-year internship · 2013

Bachelor's degree in real-time 3D
3DI Laval · 2012–2013

BTS in Visual Communication – Multimedia option
3DI Laval · 2010–2012

LANGUAGES

French (Native) · English (Technical)

Online resume
alexandre-plana.github.io/resume/

Alexandre Plana

Full-stack developer with front-end expertise

Industrial applications · UI/UX · Cross-platform

PROFILE

I develop industrial applications, drawing on a strong front-end and UI/UX background that I now extend across the full stack. On Xistar, I work from the operator interface through to the services that ingest and distribute operational data. My approach connects user needs, architecture and technical constraints, with close attention to product consistency from the initial design to delivered code.

WEB EXPERTISE AT A GLANCE

Vue 3 · micro-frontend
Three years on IECA, Schneider Electric's IIoT Edge platform: single-spa modules, shared components, Pinia stores tested with Jest and code quality tracked in SonarQube.

React · TypeScript
Xistar, a browser-based maritime command center: Dockview workspace, MapLibre and deck.gl mapping, real-time MQTT and Socket.IO feeds.

Design system
Components designed in Figma, built in React with shadcn and distributed to X-PERT applications through a private npm registry.

UX/UI & functional design
Since 2013, from business requirements to workflows and interfaces: serious games, simulators (Dosicase, Simulhom) and cross-platform applications.

VUE.JS · MICRO-FRONTEND – SCHNEIDER ELECTRIC

Schneider Electric – R&D Consultant – Datacorp — 2017 – 2026

Industrial Automation Edge (IECA) — 2023 – 2026
Front-end developer

Within a multidisciplinary Scrum team, I developed Edge applications organized as micro-frontend modules with Vue 3, TypeScript and single-spa. My work focused primarily on interfaces and front-end architecture, with occasional back-end feature extensions. I contributed to the platform's functional areas, shared components, tests and code-quality monitoring.

ACHIEVEMENTS

- Contribute to features for connectors, computations, conditions, alerts and scripting.
- Build shared components in Vue 3 and TypeScript, including a configurable table, a routing foundation and a property selector.
- Extend back-end features where required by the front-end work.
- Write unit tests for components, Pinia stores and utility functions with Jest, then monitor performance and code quality with SonarQube.
- Design an IP Sprint prototype for a mobile application that sends alerts to on-call operators outside the IECA platform.

Vue 3 · TypeScript · single-spa · Docker · Jest · HTML · CSS · Git · Figma · Jira

IECA – TECHNICAL DEEP DIVE

IECA brings together some fifteen Vue 3 applications loaded by single-spa into a common shell, alongside Go microservices built on EdgeX. Every micro-frontend shares the same foundation: Pinia, Vue Router, typed data-access services, forms validated with vee-validate and yup, and Tailwind styles isolated by prefix.

FUNCTIONAL AREAS

Connectivity
OT/IT connectors and their interfaces, acquisition groups (period, start/stop, status), variables and actions, an editable grid and variable import with an error report.

Computations
Computed variables built step by step: properties, formula, then a sliding or tumbling time window with aggregation, or an on-change trigger.

Conditions
If/Then rule editor: action type selection, data quality level and severity indicator.

Alerts & scripting
Alert table and detail view with state and activity; JavaScript scripts edited in Monaco, execution tracking and grouping of multi-line logs.

SHARED FOUNDATION

Configurable table
Generic typed component: declarative columns (property path, width, business sort order), built-in or custom cells, ascending/descending/neutral sorting, multi-selection, actions, loading skeletons and empty state.

Property selector
Search, filters by source and data type, sorting, selection limit, source/destination mode and license and write-permission checks.

Routing foundation
Routes filtered by write permissions, navigation guards and breadcrumbs declared in route metadata.

Shared library
@ieca/ieca-web-utils, published to an internal npm registry: global components, common Pinia stores, axios interceptors, error handling and shared test mocks.

Quality & delivery
The connectivity, computation, condition, alert and scripting modules and the shared library add up to nearly 300 Jest test files (Vue Test Utils, @pinia/testing, axios-mock-adapter). Every repository runs through GitHub Actions CI with SonarQube and Coverity analysis and ships as an nginx Docker image; a mock server lets each micro-frontend be developed in isolation.

Jest · Vue Test Utils · SonarQube · Coverity · GitHub Actions · Docker · pnpm

IP Sprint – mobile prototypes
Mobile application prototypes built during a 2024 IP Sprint: a mobile version of IECA data management, using Capacitor for Android and iOS and then Quasar, and an experimental mobile shell able to load single-spa micro-frontends.

Vue 3 · Capacitor · Quasar · Android · iOS · single-spa

WEB DEVELOPMENT – X-PERT

Datacorp – R&D Consultant – X-PERT — 2026

Xistar – Maritime Tactical Command Center — 2026

Full-stack developer

Xistar brings together tactical awareness, mission and drone control, alert management, multi-screen operation and mission replay. Based on operational requirements, I contribute to the functional design of operator workflows, interactions and expected feature behavior. The interface uses TypeScript, React and Dockview; mapping relies on MapLibre and deck.gl to display NATO APP-6D symbology, an MGRS grid and bathymetry. Go services ingest partner feeds through AIS, NMEA, CoT and MAVLink, transmit commands and publish information through MQTT and API REST. Deployment profiles and communication between instances allow the product to adapt to defense, port and demonstration environments.

ACHIEVEMENTS

- Contribute to Xistar's functional design based on operational requirements: define operator workflows, interactions and expected feature behavior.
- Build the TypeScript and React operator workspace around Dockview panels, then integrate MapLibre and deck.gl mapping with NATO APP-6D symbology.
- Develop the alert system and its escalation mechanisms through a dedicated API and email notifications.
- Implement interoperability in the Go services: ingest AIS, NMEA, CoT and MAVLink partner feeds, transmit commands and publish data through MQTT and API REST.
- Develop the multi-screen application and centralized control from a master workstation, then implement mission replay for analysis, situation reports and debriefings.

TypeScript React Go Dockview MapLibre deck.gl MQTT API REST

XISTAR COMPANION TOOLS

Mobile app — 2026

TypeScript developer / Mobile application

Developed a mobile application that lets operators receive BAM/CAM alerts and issue commands from the field. Built in TypeScript with React, it supports alert escalation and return orders and communicates with Xistar through Socket.IO.

Screen wall emulator — 2026

TypeScript developer / Multi-screen emulator

Developed a screen-wall emulator for designing and demonstrating Xistar multi-view workflows without an equipped control room. Built in TypeScript with React and Vite, it reproduces screen groups, a mission board and remote view dispatch from the master workstation.

Drone & naval fleet simulator — 2026

TypeScript developer / Simulation

Developed a drone and naval fleet simulator that supplies Xistar with data streams representative of operating conditions. Built in TypeScript on Node.js, it generates positions, states and detections and publishes them through MQTT for demonstrations and feature development without relying on static mock data.

X-PERT design system — from Figma to applications — 2026

UI design & front-end architecture

X-PERT's design system covers technical applications, the company website and the visual styling of official documents and videos. Components are designed in Figma, developed in TypeScript with React and shadcn, then published to a private npm registry. The library combines standard elements—cards, buttons, switches and tables—with technical interface components: charts and treemaps, activity feeds, timelines, maps, tabs and resizable panels. Internal documentation describes their integration into applications and their use in official documents.

ACHIEVEMENTS

- Design interfaces and components in Figma for technical applications and the company website.
- Build the library in TypeScript and React on top of shadcn: cards, buttons, switches, tables, charts, treemaps, activity feeds, timelines, maps, tabs and resizable panels.
- Publish components to a private npm registry for distribution across company applications.
- Define the visual styling of official documents and videos, and write internal documentation for applications and documents.

TypeScript React shadcn Figma npm

AI agent execution workflow — 2026

AI workflow design & development

I designed an AI-agent-assisted development workflow using the BMAD method as its foundation. My personal project Iasit also helped me become familiar with agent coordination, execution tracking and improvement loops, and that experience informed the project's design. Each project has its own Docker image for running agents in an isolated environment, while runs are dispatched to a dedicated machine through GitLab CI. A tool built with React and TypeScript uses agent hooks to track progress as execution unfolds. Session reports feed a self-improvement loop for both the agents and the workflow.

ACHIEVEMENTS

- Structure an AI-agent-assisted development workflow based on BMAD principles.
- Isolate agent execution in Docker containers built from a project-specific image.
- Use GitLab CI to trigger runs and dispatch them to a dedicated machine.
- Build a React and TypeScript monitoring tool that uses agent hooks to show progress as execution unfolds.
- Analyze session reports and automatically feed the lessons identified back into the agents and workflow.

BMAD Docker GitLab CI React TypeScript

AI-assisted promotional video creation workshop — 2026

R&D · Creative coding & AI

The workshop combines creative coding and generative AI to create promotional videos. The R&D work focuses on generating animations in HTML and WebGL and using HyperFrames to build animated sequences. Code serves as the medium for creating animations, with assistance from generative AI.

ACHIEVEMENTS

- Develop a workshop for creating promotional videos with generative AI.
- Explore animation generation in HTML for promotional sequences.
- Program animations and visual effects in WebGL.
- Use HyperFrames to create animations for promotional videos.

HTML WebGL HyperFrames

INDUSTRIAL AR APPLICATION — SCHNEIDER ELECTRIC

Schneider Electric — R&D Consultant — Datacorp 2017 – 2026

Augmented Operator Advisor (AOA) — Industrial AR Application 2017 – 2023

C# / Unity Developer

For six years, I developed the interfaces and front-end architecture of an industrial AR application in C# and Unity. The scope covered Android, iOS and Windows versions, customization for OEM clients, offline operation and preparation for each distribution platform. I also contributed to the project server in Python and its HTTP communication with the application.

ACHIEVEMENTS

- Design the front-end architecture and an interface adapted to Android, iOS and Windows in portrait and landscape modes using C# and Unity.
- Develop offline operation and HTTP synchronization, then contribute to the Python project server and network-error handling.
- Maintain backward compatibility by reading legacy project formats and migrating them during loading.
- Implement an externalized style system for producing custom-branded versions for OEM clients.
- Create a unit-testing tool in Unity integrated into the stages before and after builds.

Unity C# Figma Jira

SERIOUS GAMES & SIMULATION — ORECA ENGINEERING

Oreka Engineering — C# / Unity 3D Developer 2013 – 2017

Industrial XR Projects — EDF · RTE · AREVA · DCNS · SUEZ 2013 – 2017

C# / Unity 3D Developer

For four years, I contributed to the functional design and development of interactive applications for industry, nuclear operations and defense. Starting from client requirements and operational procedures, I helped define workflows, scenarios, interactions and functional mockups for training applications and simulators. I then implemented them in C# with Unity, integrated models from SolidWorks and 3ds Max, optimized CPU/GPU performance and prepared releases.

ACHIEVEMENTS

- Contribute to functional design by analyzing client requirements, assessing technical feasibility and formalizing workflows, interactions and mockups.
- Design training applications and simulators from operational procedures, then structure scenarios using state graphs, conditions and triggers.
- Develop 2D/3D interfaces and interactive systems in C# with Unity, including interactions for VR and AR devices.
- Integrate and optimize CAD models from SolidWorks and 3ds Max: meshes, levels of detail, lightmaps, materials and shaders.
- Optimize CPU/GPU performance with Unity profiling tools, then prepare builds, fix defects and support client demonstrations.

Unity C# WPF WinForms SolidWorks 3ds Max Figma

KEY PROJECTS

Dosicase — Radiation-safety training simulator 2015 – 2017

UX/UI design

Dosicase uses simulation to support practical radiation-safety training. I designed the experience and UI/UX interface to present situations and radiological risks clearly to learners, then contributed to their integration into the simulator in C# with Unity. The work covered both information architecture and the interactions required for hands-on training.

ACHIEVEMENTS

- Structure the learning journey from radiation-safety training objectives.
- Design the UI/UX interface and organize information about situations and radiological risks.
- Define the simulator interactions required for hands-on training.
- Contribute to integrating the experience into the application in C# with Unity.

Figma

Radiation protection — Nuclear safety training 2014 – 2015

C# developer / Training simulation

The module used simulated accident scenarios to teach radiation-protection and nuclear-safety procedures. I structured the training sequences, modeled the scenario flow and implemented emergency-team management. I then developed and integrated these mechanisms in C# with Unity in a 3D environment.

ACHIEVEMENTS

- Translate nuclear-safety procedures into interactive training sequences.
- Design simulated accident scenarios and their progression.
- Implement emergency-team management within the simulation.
- Develop the training module and its 3D environment in C# with Unity.

Unity C#

Musculoskeletal disorder prevention tool 2015 – 2017

Functional design & UX/UI

Simulhom is a 3D ergonomics simulator designed to analyze working postures and visualize musculoskeletal risks. I contributed to the product's functional design with ergonomists, then designed the workflows and UI/UX interface to make postural indicators clear and actionable. I also helped develop the application in C# with Unity and integrate visual feedback into the 3D scene.

ACHIEVEMENTS

- Contribute to functional design with ergonomists by formalizing use cases and analysis scenarios.
- Design workflows, the UI/UX interface and visual feedback associated with postural risks.
- Develop interactions and 3D simulator components in C# with Unity.
- Integrate postural analysis and present risk indicators throughout the simulation.

Unity C#

OTHER PROJECTS

Radiation protection for interventional surgery 2013 – 2015

C# developer / 3D medical simulation

Developed a 3D medical simulation for training interventional-surgery teams in radiation-protection practices. I contributed to representing dosimetry throughout the scenarios, visualizing radiation-exposed areas and integrating operator protocols in C# with Unity.

EMEM mobile equipment evacuation enclosure 2014 – 2016

C# developer / 3D simulation

Developed an interactive 3D simulation of EMEM, a mobile enclosure for evacuating equipment in a nuclear environment. I contributed to representing flows and radioactive materials and designed operator control interfaces using C#, Unity and WPF.

Aeroball 2014 – 2015

C# developer / Serious game

Developed a to train operators on the Aeroball control station in a nuclear environment. I helped translate operational procedures into interactive scenarios, then implemented them in C# with Unity and modeled the 3D training environment.

Steam-generator maintenance 2014 – 2015

C# developer / Serious game

Developed a to prepare personnel for steam-generator maintenance during nuclear outages. I helped translate maintenance procedures into interactive scenarios, then implemented them in C# with Unity in a 3D environment.

PERSONAL WEB PROJECTS

iasit 2026

AI-assisted development workflow

Iasit is a coordination tool for software development assisted by AI agents.

React TypeScript Node.js Zod Sigma.js API REST MCP

vorrery 2026

3D educational sandbox

Vorrery is a 3D educational sandbox designed for learning through experimentation, observation and hands-on manipulation.

React TypeScript Three.js React Three Fiber Web Workers

kin 2026

Self-hosted application

Design of a private, self-hosted family digital memory for keeping photos, videos and documents on a home server.

Node.js TypeScript Android API REST SQLite